

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 03

From Prices to Technical Indicators

Week 2 explained the observations and returns. Now ask what each indicator measures, why indicators can disagree, and whether their inputs were available before a decision.

Issue	Question	Page
01	What does a moving average measure?	3
02	Can the Close fall while its moving average rises?	4
03	How does an exponential average update?	5
04	Can MACD be negative while its histogram is positive?	6
05	Why do short and longer horizons give opposite signs?	7
06	How dispersed were the recent returns?	8
07	Does High minus Low miss an overnight gap?	9
08	How does ATR smooth True Range?	10
09	Does RSI above 70 mean the next price must fall?	11
10	More volume than yesterday, but less than usual?	12
11	Do two indicator names mean two independent inputs?	13
12	Can today's Close be used before the close?	14
13	Which later return belongs to this feature row?	15
14	Does a centered average use tomorrow's price?	16
15	Which data may set the scaling parameters?	17
16	How does a feature become a signal?	18
17	Was this filing available at the period end?	19
18	Which GDP estimate was known then?	20

Read each question, calculate from the same real prices, then explain the output. Setup is on page 2. Questions are part of the lesson, not separate submissions.

AAPL daily observations: January 2-June 28, 2024. Prices are USD/share. Returns exclude dividends and costs. Short windows illustrate arithmetic; MACD 12/26/9 and Wilder 14 are explicit settings, not recommended parameters.

This week does not rank trading performance or choose portfolio weights. Shown dates are selected teaching examples, not estimates of how frequently a pattern occurs.

DATA PREPARATION

One dataset for the whole lesson

Reuse Week 2's field and date conventions. Keep Close separate from Adj Close and run the examples in order. The short checks below stop calculation rather than silently changing prices.

Before running: Will July 1 appear when it is the exclusive end date?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

aapl = yf.Ticker("AAPL").history(
    start="2024-01-02", end="2024-07-01",
    interval="1d", auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
close = aapl["Close"]
ohlcv = aapl[["Open", "High", "Low", "Close"]]
assert aapl.index.is_unique and aapl.index.is_monotonic_increasing
assert np.isfinite(ohlcv).all().all() and (ohlcv > 0).all().all()
assert (aapl.High >= ohlcv[["Open", "Close"]].max(axis=1)).all()
assert (aapl.Low <= ohlcv[["Open", "Close"]].min(axis=1)).all()
assert np.isfinite(aapl.Volume).all() and (aapl.Volume >= 0).all()
print("Rows:", len(close), "| Dates:", close.index[0], close.index[-1])
result = pd.DataFrame({"Date": close.index[:3],
    "Close (USD/share)": close.iloc[:3].map("{:.2f}".format)})
result
```

Rows: 124 | Dates: 2024-01-02 2024-06-28

Date	Close (USD/share)
2024-01-02	185.64
2024-01-03	184.25
2024-01-04	181.91

There are 124 returned rows. January 2-4 closes are USD 185.64, 184.25 and 181.91/share. Reuse the response instead of changing the sample for each indicator.

These checks do not verify every expected exchange date or historical receipt time. The request returns today's historical version, not a log of data arriving in 2024. Missing lookback history is not zero.

Install numpy, pandas, matplotlib and yfinance if needed. All displayed code runs in one fresh Python notebook; tables are each block's final expression. Outputs below were actually executed. Formulas use rounded inputs; code retains precision.

Sources: [1], [17].

ISSUE 01

What does a moving average measure?

A simple moving average (SMA) is the mean of the prices in a trailing window. It describes an average price level, not a return or a promise of the next price.

C_t is Close in USD/share; k counts prices; j counts backward from the current row. SMA has the same USD/share unit. Use $k = 3$, with all three prices present.

$$SMA_t^{(k)} = \frac{1}{k} \sum_{j=0}^{k-1} C_{t-j}$$

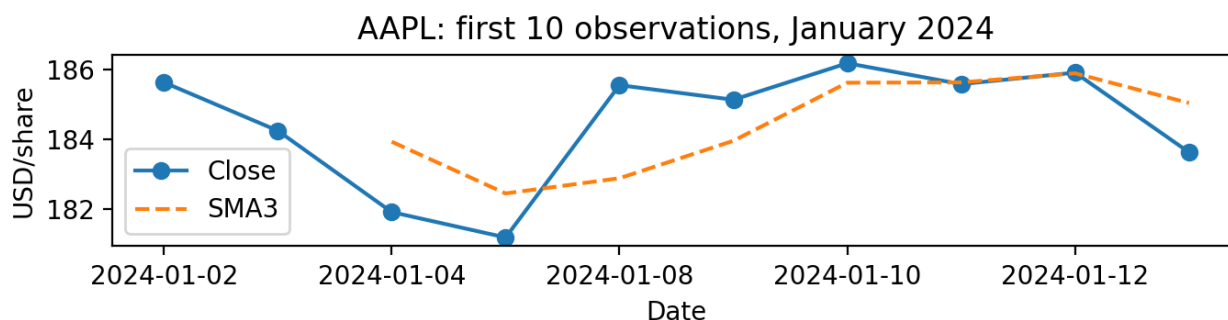
$$SMA_{Jan4}^{(3)} = \frac{185.64 + 184.25 + 181.91}{3} \approx 183.93 \text{ USD/share}$$

The first two SMA values are missing: fewer than three prices are present. `min_periods=3` does not fill them with zero.

Each price receives one third of the weight. On January 5, January 2 leaves the window and January 5 enters; the other two prices remain.

Before running: Will January 4's Close lie below its SMA3? Will the mean fall on January 5 when USD 185.64 leaves and USD 181.18 enters?

```
sma3 = close.rolling(3, min_periods=3).mean()
fig, ax = plt.subplots(figsize=(7, 2))
close.iloc[:10].plot(ax=ax, label="Close", marker="o")
sma3.iloc[:10].plot(ax=ax, label="SMA3", linestyle="--")
ax.set(title="AAPL: first 10 observations, January 2024",
       xlabel="Date", ylabel="USD/share")
ax.legend()
plt.tight_layout()
```



January 4's Close, USD 181.91/share, is below its USD 183.93 mean. The mean falls to USD 182.45 on January 5. Smoothing past prices does not show that the next price will rise toward the mean.

Discuss

1. Calculate January 5's mean using January 3-5 prices. Which observation leaves the preceding window?
2. Does a Close below its SMA establish a profitable buying opportunity? Name the later outcome and execution evidence still needed.

Sources: [3], [10]. Full titles and links are on the references page.

ISSUE 02

Can the Close fall while its moving average rises?

A moving average can rise on a day when the closing price falls. It compares the entering price with the price leaving the window, not just with yesterday.

For the three-price SMA, C means Close in USD/share. Subtract yesterday's average from today's: two shared prices cancel, leaving the new Close minus the Close three observations earlier.

$$SMA_t^{(3)} - SMA_{t-1}^{(3)} = \frac{C_t - C_{t-3}}{3}$$

On January 9, Close fell from January 8's USD 185.56 to USD 185.14/share. But January 4's USD 181.91 leaves the three-price window.

$$SMA_{Jan9}^{(3)} - SMA_{Jan8}^{(3)} = \frac{185.14 - 181.91}{3} \approx 1.0767 \text{ USD/share}$$

Before running: Can the Close column fall while the SMA3 column rises on January 9?

```
dates = ["2024-01-08", "2024-01-09"]
result = pd.DataFrame({"Date": dates,
    "Close (USD)": [f"{close[d]:.2f}" for d in dates],
    "SMA3 (USD)": [f"{sma3[d]:.4f}" for d in dates]})
result
```

Date	Close (USD)	SMA3 (USD)
2024-01-08	185.56	182.8833
2024-01-09	185.14	183.9600

Close falls USD 0.42/share, but SMA3 rises from USD 182.8833 to USD 183.9600/share. The new price exceeds the discarded price.

Neither column is wrong. The daily change compares two adjacent closes; the change in SMA depends on the entering and leaving observations. A rising SMA alone does not mean today was an up day.

Discuss

1. Write the three dates in each average. Which two cancel when the averages are subtracted?
2. If a headline says the stock rose because its SMA rose, how would you correct it using the January 8-9 table?

Sources: [3], [10]. Full titles and links are on the references page.

ISSUE 03

How does an exponential average update?

An exponential moving average (EMA) updates the previous average using the newest price. Unlike a three-price SMA, it does not completely drop prices older than three rows.

E_t is the EMA and C_t the Close, both USD/share. Alpha is the newest price's weight. Use $\text{span} = 3$, $\alpha = 0.5$, $\text{adjust}=\text{False}$, and seed $E_0 = \text{January 2 Close}$. $\text{min_periods}=3$ hides the first two outputs, not the calculations.

$$E_t = \alpha C_t + (1 - \alpha)E_{t-1}, \quad \alpha = \frac{2}{\text{span} + 1}$$

$$\alpha = \frac{2}{3+1} = 0.5; \quad E_{\text{Jan3}} = 0.5(184.25) + 0.5(185.64) = 184.945$$

$$E_{\text{Jan4}} = 0.5(181.91) + 0.5(184.945) = 183.4275 \text{ USD/share}$$

The January 4 value still includes January 2 indirectly through January 3's average. These are weights on past observations, not portfolio allocations.

Before running: Does hiding the first two EMA outputs mean the recursion starts on January 4? Check the hand calculation.

```
ema3 = close.ewm(span=3, adjust=False,
min_periods=3, ignore_na=False).mean()
result = pd.DataFrame({"Date": close.index[2:5],
"SMA3 (USD)": sma3.iloc[2:5].map("{:.4f}".format),
"EMA3 (USD)": ema3.iloc[2:5].map("{:.4f}".format)})
result
```

Date	SMA3 (USD)	EMA3 (USD)
2024-01-04	183.9333	183.4275
2024-01-05	182.4467	182.3037
2024-01-08	182.8833	183.9319

On January 4 the EMA is USD 183.4275/share, versus SMA USD 183.9333. The same prices produce different averages because the weights differ. This is not a prediction contest.

Discuss

1. Expand January 4's recursion to find the weights on January 2, 3 and 4. Do they sum to one?
2. Why must a classmate state the seed and adjust setting before comparing their EMA with yours? Does $\text{span}=3$ mean exactly three prices are used?

Sources: [4], [11]. Full titles and links are on the references page.

ISSUE 04

Can MACD be negative while its histogram is positive?

Moving Average Convergence Divergence (MACD) compares two exponential moving averages. Its histogram compares MACD with a smoothed MACD line. These are different comparisons.

E with superscripts 12 and 26 are Close EMAs with those spans. MACD is their difference; S is its 9-span EMA, called the signal line. These values and the histogram are USD/share, not returns. Each price EMA starts at January 2 Close; S starts at the first MACD value, zero. Earlier initialization can change values.

$$\text{MACD}_t = E_t^{(12)} - E_t^{(26)}$$

$$\text{Histogram}_t = \text{MACD}_t - S_t$$

$$\text{MACD}_{\text{Apr25}} = 168.86346 - 170.47263 \approx -1.6092$$

$$\text{Histogram}_{\text{Apr25}} \approx -1.60917 - (-1.77484) \approx 0.1657 \text{ USD/share}$$

Before running: Can MACD be negative while its histogram is positive? Check April 25.

```
ema12 = close.ewm(span=12, adjust=False).mean()
ema26 = close.ewm(span=26, adjust=False).mean()
macd = ema12 - ema26
signal_line = macd.ewm(span=9, adjust=False).mean()
histogram = macd - signal_line
dates = ["2024-04-24", "2024-04-25"]
result = pd.DataFrame({"Date": dates,
    "MACD (USD)": [f"{macd[d]:.4f}" for d in dates],
    "Signal (USD)": [f"{signal_line[d]:.4f}" for d in dates],
    "Histogram (USD)": [f"{histogram[d]:.4f}" for d in dates]})
result
```

Date	MACD (USD)	Signal (USD)	Histogram (USD)
2024-04-24	-1.8424	-1.8163	-0.0262
2024-04-25	-1.6092	-1.7748	0.1657

On April 25 MACD is below zero but above its signal line. The positive histogram means the second comparison is positive; it does not mean the 12-span price EMA exceeds the 26-span EMA.

Discuss

1. For April 25, which two averages does negative MACD compare? Which two values does the positive histogram compare?
2. Does the histogram crossing zero establish a profitable entry? Name the execution rule and later outcome still absent.

Sources: [4], [18]. Full titles and links are on the references page.

ISSUE 05

Why do short and longer horizons give opposite signs?

The same stock can show a positive short-horizon return and a negative longer-horizon return on the same date. State the horizon before calling its momentum positive.

C is Close in USD/share; h counts observation intervals. Momentum m is the decimal price return from t-h to t. On January 9, the three-interval start is January 4; the five-interval start is January 2.

$$m_t^{(h)} = \frac{C_t}{C_{t-h}} - 1$$

$$m_{\text{Jan9}}^{(3)} = \frac{185.14}{181.91} - 1 \approx 1.7756\%, \quad m_{\text{Jan9}}^{(5)} = \frac{185.14}{185.64} - 1 \approx -0.2693\%$$

Before running: Will January 9 have opposite signs in the two columns even though they use the same ending Close?

```
momentum3 = close / close.shift(3) - 1
momentum5 = close / close.shift(5) - 1
dates = ["2024-01-09", "2024-01-10", "2024-01-11"]
result = pd.DataFrame({"Date": dates,
    "3 intervals": [f"{momentum3[d]:.4%}" for d in dates],
    "5 intervals": [f"{momentum5[d]:.4%}" for d in dates]})
result
```

Date	3 intervals	5 intervals
2024-01-09	1.7756%	-0.2693%
2024-01-10	2.7652%	1.0529%
2024-01-11	0.0162%	2.0230%

January 9 returns are 1.7756% and -0.2693%. Price has recovered above January 4 but remains below January 2. No contradictory measurement is involved.

Only the starting observation changes; the stock, ending date and Close convention remain fixed. Choosing the horizon after seeing which sign is preferred changes the question after seeing the answer.

Discuss

1. Use the three dated closes to explain how both January 9 results can be correct. Which period does each describe?
2. Would choosing whichever horizon has the higher return demonstrate predictive skill? State what must be fixed before testing later returns.

Sources: [5], [13]. Full titles and links are on the references page.

ISSUE 06

How dispersed were the recent returns?

Volatility describes dispersion in returns, not whether prices rose overall.

r_i are the three decimal close-to-close returns ending January 3, 4 and 5; $\bar{r}$ is their mean. With $n=3$ returns, sample standard deviation sigma uses $n-1=2$ in the denominator. It has decimal-return units, not USD/share.

$$\bar{r} = \frac{r_1 + r_2 + r_3}{3}, \quad \hat{\sigma} = \sqrt{\frac{(r_1 - \bar{r})^2 + (r_2 - \bar{r})^2 + (r_3 - \bar{r})^2}{2}}$$

$$\bar{r} = \frac{-0.0074876 - 0.0127001 - 0.0040130}{3} \approx -0.0080669$$

$$\hat{\sigma} \approx \sqrt{\frac{(0.0005793)^2 + (-0.0046332)^2 + (0.0040539)^2}{2}} \approx 0.004372 = 0.4372\%$$

Subtract the mean, square each deviation, divide their sum by 2, then take the square root. Squaring avoids cancellation of positive and negative deviations. The square root restores the return unit.

Before running: Will the January 5 rolling value reproduce the three-return hand calculation? Does its positive sign mean a positive price return?

```
daily_return = close.pct_change(fill_method=None)
vol3 = daily_return.rolling(3, min_periods=3).std(ddof=1)
dates = ["2024-01-05", "2024-01-08"]
result = pd.DataFrame({"Ending date": dates,
    "3-return volatility": [f"{vol3[d]:.4%}" for d in dates]})
result
```

Ending date	3-return volatility
2024-01-05	0.4372%
2024-01-08	1.9278%

The January 5 sample standard deviation is 0.4372% per interval. Its sign is nonnegative even though all three daily returns were negative.

Discuss

1. Why can three negative returns have positive standard deviation? Explain centering, squaring and the square root.

2. Why do three returns need four prices? What history changes in a 20-return window?

Sources: [3], [14]. Full titles and links are on the references page.

ISSUE 07

Does High minus Low miss an overnight gap?

High minus Low measures only today's session range. True Range also measures distances from the preceding Close, so it can reveal a gap omitted by the session range.

H, L and C are High, Low and Close in USD/share. TR is the largest of three nonnegative distances. On January 4, H=183.09, L=180.88 and January 3 Close=184.25. The previous Close lies above the entire January 4 range.

$$TR_t = \max\{H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|\}$$

$$TR_{\text{Jan4}} = \max\{183.09 - 180.88, |183.09 - 184.25|, |180.88 - 184.25|\} = 3.37$$

The USD 2.21 range misses the additional distance from yesterday's Close. TR is USD 3.37/share, but it still gives no direction or executable buy-and-sell return.

Before running: On which displayed date will True Range exceed High minus Low?

```
previous_close = close.shift(1)
distances = pd.concat([
    aapl["High"] - aapl["Low"],
    (aapl["High"] - previous_close).abs(),
    (aapl["Low"] - previous_close).abs()], axis=1)
# Omit Jan 2: its preceding Close is outside this sample.
true_range = distances.iloc[1:].max(axis=1)
dates = ["2024-01-03", "2024-01-04"]
result = pd.DataFrame({"Date": dates,
    "High-Low (USD)": [f"{distances.loc[d,0]:.2f}" for d in dates],
    "True Range (USD)": [f"{true_range[d]:.2f}" for d in dates]})
result
```

Date	High-Low (USD)	True Range (USD)
2024-01-03	2.45	2.45
2024-01-04	2.21	3.37

January 3 has the same USD 2.45 value for both measurements. January 4 has USD 2.21 versus USD 3.37. We changed the measurement, not the OHLC data.

Discuss

1. Why does the previous Close matter on January 4? Identify the distance selected by max and explain its USD/share unit.

2. Can TR tell whether the stock rose or fell? Which signed price comparison would answer that different question?

Sources: [21]. Full titles and links are on the references page.

ISSUE 08

How does ATR smooth True Range?

Average True Range (ATR) smooths True Range across observations. A smaller new TR can lower ATR even though all TR values are positive.

ATR and TR are USD/share. Use Wilder's 14-observation smoothing: seed ATR with the mean of 14 valid TR values, then give each new TR weight 1/14. The January 3-23 inputs sum to USD 46.9100/share; January 2 was omitted. A chart using earlier history can differ.

$$ATR_t = \frac{13 ATR_{t-1} + TR_t}{14}$$

$$ATR_{Jan23} \approx 46.9100/14 = 3.3507 \text{ USD/share}$$

$$ATR_{Jan24} \approx \frac{13(3.3507) + 2.0400}{14} = 3.2571 \text{ USD/share}$$

Before running: Will January 24 TR of USD 2.0400 pull the preceding USD 3.3507 ATR up or down?

```
def wilder_mean(values, n=14):
    assert len(values) >= n and np.isfinite(values).all()
    smoothed = pd.Series(np.nan, index=values.index)
    smoothed.iloc[n-1] = values.iloc[:n].mean()
    for i in range(n, len(values)):
        smoothed.iloc[i] = (
            (n-1)*smoothed.iloc[i-1] + values.iloc[i]) / n
    return smoothed

atr14 = wilder_mean(true_range)
dates = ["2024-01-23", "2024-01-24"]
result = pd.DataFrame({"Date": dates,
    "TR (USD)": [f"{true_range[d]:.4f}" for d in dates],
    "ATR14 (USD)": [f"{atr14[d]:.4f}" for d in dates]})
result
```

Date	TR (USD)	ATR14 (USD)
2024-01-23	1.9200	3.3507
2024-01-24	2.0400	3.2571

ATR falls from USD 3.3507 to USD 3.2571/share. The 13/14 weight retains the previous average; this is not a fresh simple mean of the latest 14 values.

ATR is not percent return volatility. Comparing raw dollar ATR across differently priced stocks is not a comparison on the same percentage scale.

Discuss

1. Why does a positive new TR lower ATR on January 24? Compare the new value with the previous average, not with zero.
2. Why would changing the initial history or replacing Wilder smoothing with a rolling mean change the result?

Sources: [19]. Full titles and links are on the references page.

ISSUE 09

Does RSI above 70 mean the next price must fall?

The Relative Strength Index (RSI) compares upward and downward price changes. A value above 70 is called overbought; it does not mean the next price must fall.

U is the positive part of a daily Close change; D is the magnitude of its negative part, both USD/share. Smooth each with the same Wilder method as Issue 8, starting with the first 14 changes. Their smoothed values are U-bar and D-bar. RSI is on a 0-100 scale, not percent return.

$$RSI_t = 100 \frac{\bar{U}_t}{\bar{U}_t + \bar{D}_t}$$

On June 11, U-bar is USD 1.995986/share and D-bar USD 0.657589/share. RSI measures the upward share of their sum.

$$RSI_{\text{Jun11}} = 100 \frac{1.995986}{1.995986 + 0.657589} \approx 75.22$$

Before running: Does the Close fall immediately after the displayed June 11 RSI exceeds 70?

```
change = close.diff().iloc[1:]
up = wilder_mean(change.clip(lower=0))
down = wilder_mean((-change).clip(lower=0))
total = up + down
# Both zero: undefined here; otherwise endpoints can be 0 or 100.
rsi14 = 100 * up / total.where(total > 0)
dates = ["2024-06-11", "2024-06-12", "2024-06-13"]
result = pd.DataFrame({"Date": dates,
    "Close (USD)": [f"{close[d]:.2f}" for d in dates],
    "RSI14": [f"{rsi14[d]:.2f}" for d in dates]})
result
```

Date	Close (USD)	RSI14
2024-06-11	207.15	75.22
2024-06-12	213.07	78.85
2024-06-13	214.24	79.49

June 11 RSI is 75.22, yet Close rises from USD 207.15 to USD 213.07/share on June 12 and USD 214.24 on June 13. This selected counterexample rejects an immediate-fall guarantee; it does not estimate how often price rises after RSI exceeds 70.

Later closes test this claim; they are not inputs to the earlier RSI.

Discuss

1. Why does RSI 75.22 not mean a 75.22% gain? Identify the dollar quantities whose ratio creates this scale.
2. Does this three-date example prove that buying above RSI 70 works? State what later evaluation would still be required.

Sources: [20]. Full titles and links are on the references page.

ISSUE 10

More volume than yesterday, but less than usual?

Volume can increase from yesterday while remaining below its recent average. The phrase higher volume is incomplete until we state the comparison.

Use February 7: 53,439,000 shares, versus February 6's 43,490,800. The mean of the preceding 20 observations is 56,683,825 shares. Exclude the current observation from that reference mean.

Vol means daily shares traded. v is the current Volume divided by the preceding 20-observation mean: a dimensionless multiple, not percent return.

$$V_t = \frac{\text{Vol}_t}{\frac{1}{20} \sum_{j=1}^{20} \text{Vol}_{t-j}}, \quad V_{\text{Feb7}} = \frac{53,439,000}{56,683,825} \approx 0.943$$

Before running: Can February 7 show an increase versus yesterday and a multiple below 1 versus the prior average?

```
volume = aapl["Volume"]
prior20 = volume.shift(1).rolling(20, min_periods=20).mean()
assert (prior20.dropna() > 0).all()
volume_multiple = volume / prior20
daily_volume_change = volume / volume.shift(1) - 1
dates = ["2024-02-07", "2024-02-09"]
result = pd.DataFrame({"Date": dates,
    "Change vs yesterday": [f"{daily_volume_change[d]:.2%}" for d in dates],
    "Vs prior 20 mean": [f"{volume_multiple[d]:.3f}x" for d in dates]})
result
```

Date	Change vs yesterday	Vs prior 20 mean
2024-02-07	22.87%	0.943x
2024-02-09	10.24%	0.793x

February 7 volume is 22.87% above yesterday but only 0.943 times the preceding average. The first statement is a one-day comparison; the second places activity in a longer history.

A ratio above 1 does not identify who traded, why they traded, or whether price will rise. It is also not an executable order-size limit.

Discuss

1. Why does the code shift Volume before taking its average? Which dates should the denominator exclude?
2. Rewrite February 7 had high volume as two numerical statements, one for each baseline. Do either establish buying pressure?

Sources: [3], [15]. Full titles and links are on the references page.

ISSUE 11

Do two indicator names mean two independent inputs?

Several indicators can give the same direction because they reuse the same prices. Agreement is not automatically several independent pieces of evidence.

For AAPL on April 25, compare the 12/26-span EMA difference with MACD. These are not merely related: Issue 4 defines them as the same calculation. The histogram asks a different comparison but still derives from MACD.

Before running: Will the first two numerical columns disagree on either date? Why must they match?

```
dates = ["2024-04-24", "2024-04-25"]
result = pd.DataFrame({"Date": dates,
    "EMA12-EMA26 (USD)": [f"{{ema12-ema26}}[d]:.4f}" for d in dates],
    "MACD (USD)": [f"{{macd}}[d]:.4f}" for d in dates],
    "Histogram (USD)": [f"{{histogram}}[d]:.4f}" for d in dates]})
result
```

Date	EMA12-EMA26 (USD)	MACD (USD)	Histogram (USD)
2024-04-24	-1.8424	-1.8424	-0.0262
2024-04-25	-1.6092	-1.6092	0.1657

Both columns are -1.6092 USD/share on April 25. Counting their negative signs as two independent confirmations counts one calculation twice. The +0.1657 histogram is a different subtraction, not an independent data source.

Before adding a feature, name its inputs, transformation and question. Volume provides a different observed variable; whether it improves prediction is a later empirical question, not something its different name proves.

Discuss

1. A report claims two indicators independently confirm a decline using the first two columns. Which identity invalidates that claim?
2. Would adding the histogram establish an independent third source? Explain its dependence on MACD without claiming that all derived features are useless.

Sources: [12], [18]. Full titles and links are on the references page.

ISSUE 12

Can today's Close be used before the close?

A final daily Close cannot be an input to a decision made before that session has ended. A correct price with the wrong availability assumption still uses future information.

On January 4, 2024, AAPL closed at USD 181.91/share. Nasdaq's regular session ends at 16:00 New York time. The earlier USD 184.25 close belongs to January 3. A 15:00 January 4 decision cannot already know the final January 4 Close.

Date	Close (USD/share)	Known by Jan 4, 15:00?
Jan 3, 2024	184.25	Earlier completed session
Jan 4, 2024	181.91	No: session not finished

At 15:00, the current final Close has not occurred. After 16:00, use it only after the completed row has actually arrived and the calculation has finished. The daily history request does not provide that historical receipt time.

A feature using Close, High, Low and Volume must wait for all of its required final daily fields. An early provisional row is not automatically the same observation as the completed historical row.

Discuss

1. A rule claims to use the final January 4 Close at 15:00 that day. Which exact input is unavailable, and can the January 3 Close substitute without changing the rule?
2. An analyst receives Close before final Volume. May a feature requiring both be computed as if both were final? State what must arrive first.

Sources: [1], [2]. Full titles and links are on the references page.

ISSUE 13

Which later return belongs to this feature row?

A feature describes available inputs; a target is the later outcome we want to predict. Storing them on the same row does not make both known at the same time.

C_t is Close on the feature date and C_{t+1} the next observation's Close, both USD/share. y_t is the next-interval price-return target, in decimal units. For $t = \text{January 4}$, the target ends January 5.

$$y_t = \frac{C_{t+1}}{C_t} - 1$$

$$y_{\text{Jan4}} = \frac{181.18}{181.91} - 1 \approx -0.004013 = -0.4013\%$$

January 4's row may store this target for later analysis, but USD 181.18/share is January 5's Close. It was not available with January 4's completed row.

Before running: Why must the June 28 target remain missing in a sample ending June 28?

```
target = close.shift(-1) / close - 1
dates = ["2024-01-04", "2024-01-05", "2024-06-28"]
result = pd.DataFrame({"Feature date": dates,
    "Next-return target": ["Missing" if pd.isna(target[d])
        else f"{target[d]:.4%}" for d in dates]})
result
```

Feature date	Next-return target
2024-01-04	-0.4013%
2024-01-05	2.4175%
2024-06-28	Missing

January 4's target is -0.4013%. June 28 has no later Close in the sample, so its target is missing. A negative shift changes row placement, not when the outcome becomes known.

Training at a cutoff requires targets already observed by that cutoff. An after-close signal also cannot assume a fill at the already-observed Close; target prediction and earned trading return are different tasks. Week 4 defines execution and holdings.

Discuss

1. Would using target as a January 4 feature introduce future information? Name the price and date responsible.
2. At a June 28 training cutoff, may you use a subsequently downloaded July 1 Close to label June 28 as if that target were already known? Explain the timing.

Sources: [5], [6]. Full titles and links are on the references page.

ISSUE 14

Does a centered average use tomorrow's price?

A centered moving average can place tomorrow's price into today's row. This can describe a completed historical chart, but it cannot be a feature available today.

For t = January 4, C means Close in USD/share. A trailing three-price mean uses January 2, 3 and 4. A centered three-price mean instead uses January 3, 4 and 5.

$$SMA_t^{(3)} = \frac{C_{t-2} + C_{t-1} + C_t}{3}$$

$$SMA_{\text{Jan4}}^{(3)} = \frac{185.64 + 184.25 + 181.91}{3} \approx 183.9333$$

For comparison, the centered mean includes C_{t+1} , a later price. Both means have USD/share units.

$$\text{Centered mean}_t = \frac{C_{t-1} + C_t + C_{t+1}}{3} = \frac{184.25 + 181.91 + 181.18}{3} \approx 182.4467$$

Before running: If the same real dataset is restricted to January 4 and earlier, can the centered value on January 4 still be calculated?

```
centered3 = close.rolling(3, min_periods=3, center=True).mean()
through_jan4 = close.loc[:"2024-01-04"]
prefix_centered = through_jan4.rolling(3, center=True).mean()
result = pd.DataFrame({"Date": ["2024-01-04"],
    "Trailing (USD)": [f"{sma3['2024-01-04']:.4f}"],
    "Centered (USD)": [f"{centered3['2024-01-04']:.4f}"],
    "Centered without Jan 5": ["Missing" if pd.isna(
        prefix_centered['2024-01-04']) else "Available"]})
result
```

Date	Trailing (USD)	Centered (USD)	Centered without Jan 5
2024-01-04	183.9333	182.4467	Missing

With all dates, the centered value is USD 182.4467/share. Restricting the same data to January 4 removes its required later input, so that centered value is missing. No price was fabricated or changed.

Discuss

1. Which exact price enters the centered January 4 value but not the trailing one? Why is the lower centered value not evidence of better prediction?
2. Recalculate a trailing mean using only data through its labeled date. What should stay unchanged, and what would an unexpected difference make you inspect?

Sources: [3], [6]. Full titles and links are on the references page.

ISSUE 15

Which data may set the scaling parameters?

Scaling parameters must be estimated before the later observations they transform. Using the full file can silently place future information into an earlier standardized feature.

$m_t^{(3)}$ is three-interval momentum as a decimal. The fitted mean is μ_{fit} ; σ_{fit} is the estimated sample standard deviation (ddof=1), marked with a hat in the formula. z_t is dimensionless. Fit through April 4, 2024, then freeze the parameters for April 5. The standard deviation must be positive and finite.

$$z_t = \frac{m_t^{(3)} - \mu_{\text{fit}}}{\hat{\sigma}_{\text{fit}}}$$

$$z_{\text{Apr5}} = \frac{0.0043829 - (-0.0038024)}{0.0227433} \approx 0.3599$$

The fitted parameters use 62 momentum observations through April 4. The April 5 input is 0.0043829, the earlier mean -0.0038024, and earlier sample standard deviation 0.0227433. Do not estimate them again using later June observations.

Before running: Which column uses observations unavailable for an April 5 decision? Are the two columns transforming the same input?

```
fit = momentum3.loc["2024-04-04"].dropna()
mu, scale = fit.mean(), fit.std(ddof=1)
assert np.isfinite(scale) and scale > 0
apr5 = momentum3.loc["2024-04-05"]
z_earlier = (apr5 - mu) / scale
z_full = (apr5 - momentum3.mean()) / momentum3.std(ddof=1)
result = pd.DataFrame({"Date": ["2024-04-05"],
    "Fit through Apr 4": [f"{z_earlier:.4f}"],
    "Fit through Jun 28": [f"{z_full:.4f}"]})
result
```

Date	Fit through Apr 4	Fit through Jun 28
2024-04-05	0.3599	0.0150

The same April 5 momentum becomes 0.3599 under the earlier fit and 0.0150 under the full-file fit. These are standardized units, not percent returns. Only the fitting window changed.

The April 4 fit can be used later after data receipt and computation. It was not already known at every earlier date inside its own training sample. This sample-standard-deviation convention also differs from some software defaults.

Discuss

1. Why is the full-file result invalid for an April 5 real-time feature even if it is numerically close to the earlier-fit result?
2. Can the April 4 parameters be assigned to a January 5 historical decision as if already known then? Distinguish offline training from a contemporaneous decision.

Sources: [6], [14]. Full titles and links are on the references page.

ISSUE 16

How does a feature become a signal?

A signal applies an explicit rule to a feature. Missing history must remain different from a valid zero. The signal does not yet specify shares, weights or trades.

Use three-interval momentum $m_t^{(3)}$. Our teaching rule sets signal s_t to 1 if momentum is positive, 0 if it is zero or negative, and Missing if the feature is unavailable. These are rule settings, not market observations.

Feature condition	Signal	Meaning
Positive	1	Positive past-return condition
Zero or negative	0	Condition not met
Unavailable	Missing	Rule cannot be evaluated

January 5 momentum is -2.4025%, so the signal is 0. January 9 momentum is 1.7756%, so it is 1. January 4 has no three-interval starting price in the sample: its signal stays Missing.

Before running: Will a comparison `momentum3 > 0` alone preserve the difference between missing momentum and a nonpositive observation?

```
signal = pd.Series(np.nan, index=close.index)
valid = momentum3.notna()
assert np.isfinite(momentum3[valid]).all()
signal.loc[valid] = (momentum3.loc[valid] > 0).astype(int)
dates = ["2024-01-04", "2024-01-05", "2024-01-09"]
result = pd.DataFrame({"Date": dates,
    "Momentum": ["Missing" if pd.isna(momentum3[d])
        else f"{momentum3[d]:.4%}" for d in dates],
    "Signal": ["Missing" if pd.isna(signal[d]) else str(int(signal[d]))
        for d in dates]})
result
```

Date	Momentum	Signal
2024-01-04	Missing	Missing
2024-01-05	-2.4025%	0
2024-01-09	1.7756%	1

The outputs are Missing, 0 and 1 for the three dates. A bare comparison treats a NaN comparison as false; the explicit valid mask prevents that false result from becoming a measured signal 0.

For January 9, the three-interval input uses January 4 Close, USD 181.91/share, and January 9 Close, USD 185.14/share. Signal 1 still does not specify an execution price, quantity or holding interval; Week 4 defines those.

Discuss

1. Why is filling missing momentum with zero a change to the rule rather than a recovery of data?
2. For January 9, identify the two source prices, the feature sign and the resulting signal. What execution information is still needed before claiming a trading return?

Sources: [5], [6], [13]. Full titles and links are on the references page.

ISSUE 17

Was this filing available at the period end?

A financial report describes an earlier period but is filed later. Joining its contents to the period-end date can put an unavailable document into earlier decisions.

Apple's fiscal 2024 first-quarter Form 10-Q has a period of report ending December 30, 2023. The SEC record separately lists an accepted timestamp of February 1, 2024 at 18:03:38 and a filing date of February 2.

Company / form	Period end	Accepted	Filing date
Apple 10-Q	2023-12-30	2024-02-01 18:03:38	2024-02-02

December 30 tells us the period described. It does not mean this February filing was available on December 30. Nor does acceptance prove the exact time every analyst received and processed it.

For timestamp matching only, compare two hypothetical February 1 decisions at 16:00 and 18:30 using the same clock convention as the SEC display. This assumption avoids inventing a timezone conversion; matching still does not establish actual receipt.

Before running: Which hypothetical decision precedes the displayed acceptance timestamp?

```
accepted = pd.Timestamp("2024-02-01 18:03:38")
decision_times = pd.to_datetime([
    "2024-02-01 16:00:00", "2024-02-01 18:30:00"])
result = pd.DataFrame({"Decision (same clock)":
    decision_times.strftime("Feb 1 %H:%M"),
    "After SEC acceptance": decision_times > accepted})
result
```

Decision (same clock)	After SEC acceptance
Feb 1 16:00	False
Feb 1 18:30	True

The 16:00 row is False and the 18:30 row True. True means only that acceptance is earlier than the stipulated decision clock, not that the filing was delivered in time for trading.

Discuss

1. Why would assigning this filing to every date after December 30 create a timing error? Does the period-end label identify its publication?
2. What additional evidence is needed before the 18:30 timestamp match becomes a valid information-availability claim? Consider dissemination, receipt and processing.

Sources: [7]. Full titles and links are on the references page.

ISSUE 18

Which GDP estimate was known then?

The same economic period can have several published estimates. A later revised number cannot replace the value that was known at an earlier date.

BEA initially reported Q4 2023 real GDP growth of 3.3% on January 25, 2024. Its February 28 second estimate was 3.2%. Both are seasonally adjusted annualized growth from the preceding quarter, not year-over-year growth or an investment return.

Release date	Quarter	Growth (%)
2024-01-25	2023 Q4	3.3
2024-02-28	2023 Q4	3.2

The revision is $3.2 - 3.3 = -0.1$ percentage point. The earlier estimate was not a data-entry error simply because BEA later revised it. These two historical releases are retained intentionally; newer estimates exist.

Before running: Using only releases earlier than each hypothetical decision date, should February 1 and March 1 receive the same estimate?

```
# Values transcribed from the two dated BEA releases.
releases = pd.DataFrame({"Release": pd.to_datetime(
    ["2024-01-25", "2024-02-28"]), "GDP (%)": [3.3, 3.2]})
decisions = pd.DataFrame({"Decision": pd.to_datetime(
    ["2024-02-01", "2024-03-01"])})
matched = pd.merge_asof(decisions, releases,
    left_on="Decision", right_on="Release", direction="backward")
result = matched.copy()
for field in ["Decision", "Release"]:
    result[field] = result[field].dt.strftime("%Y-%m-%d")
result
```

Decision	Release	GDP (%)
2024-02-01	2024-01-25	3.3
2024-03-01	2024-02-28	3.2

With dates sorted, `direction="backward"` picks the latest release at or before each decision. February 1 receives 3.3%; March 1 receives 3.2%. Both are later than the release day; no same-day timing assumption is needed. Using 3.2% in both rows would import a later revision into February 1.

Together with Issue 17, retain the period described, release date and version. Week 4 can then connect features and signals to execution; none of these calculations establishes a profitable trading rule.

Discuss

1. Why is -0.1 percentage point a revision in a growth estimate, not a 0.1% investment loss?
2. For a February 1 historical study, which estimate belongs in the inputs? Explain why a more recent download may be unsuitable without vintage information.

Sources: [8], [9], [16]. Full titles and links are on the references page.

REFERENCES & REPRODUCTION

Sources and calculation notes

- [1] [yfinance historical-data interface](#)
- [2] [Nasdaq market hours](#)
- [3] [pandas rolling windows](#)
- [4] [pandas exponentially weighted calculations](#)
- [5] [pandas shift](#)
- [6] [scikit-learn: data leakage](#)
- [7] [Apple Form 10-Q filing record](#)
- [8] [BEA: Q4 2023 GDP advance estimate](#)
- [9] [BEA: Q4 2023 GDP second estimate](#)
- [10] [Fidelity: simple moving average](#)
- [11] [Fidelity: exponential moving average](#)
- [12] [Lo, Mamaysky and Wang: technical analysis research](#)
- [13] [Fidelity: rate of change](#)
- [14] [NIST: standard deviation](#)
- [15] [Nasdaq glossary: volume](#)
- [16] [pandas: match earlier observations](#)
- [17] [Yahoo Finance: AAPL history and price-field notes](#)
- [18] [Fidelity: MACD](#)
- [19] [StockCharts: ATR calculation and initialization](#)
- [20] [StockCharts: RSI calculation and interpretation](#)
- [21] [Fidelity: True Range and ATR](#)

How the sources are used

Market observations come from Yahoo Finance through `yfinance`. Only selected inputs and derived results are shown; the full download is not redistributed. SEC and BEA support dated document and release facts. Software documentation supports calculation behavior, not prediction or profitability.

SMA, EMA and rate-of-change sources identify established indicators. Our short-window examples are not recommendations or replications of the Lo, Mamaysky and Wang research design. Figures describe observed data only.

Executed 2026-09-19 UTC: Python 3.12.9, numpy 2.2.6, pandas 2.2.3, matplotlib 3.10.8, yfinance 1.2.0. Run all displayed code in order in a fresh Python notebook. Install the named packages if absent. Data access requires internet and provider availability.

Historical responses may change. Receipt times and point-in-time market-data versions were not independently verified. The checked calculations do not fill those evidentiary gaps. This lesson does not assign weights, fills, costs or trading performance.